Myoperi_following_COVID19vaccination_2022_2025

Author: Godwin Anguzu

Study Overview

Myopericarditis -- encompassing myocarditis (inflammation of the heart muscle) and pericarditis (inflammation of the pericardial sac) -- emerged as a recognized adverse event of special interest (AESI) following mRNA-based COVID-19 vaccination, particularly in adolescent males and young adults. Although the absolute risk is low, understanding its incidence in real-world populations is critical for informing vaccination guidelines, safety monitoring, and clinical decision-making.

Objectives

This analysis uses longitudinal claims data from 2022 to 2025 to:

1. **Estimate crude incidence rates** of myopericarditis (AESI code: myo1) in a cohort of vaccinated individuals aged 12-24, expressed as events per 100,000 person-days at risk during the risk window following each vaccine dose.

2. **Stratify by vaccine brand and age group** to identify subgroups that may carry disproportionate burden, consistent with prior signal detection findings for mRNA-1273 (Moderna) and BNT162b2 (Pfizer-BioNTech).

Analytic Approach

The pipeline proceeds in two stages:

Stage 1 -- Data Wrangling (Parts 1-5): Starting from raw administrative files (event, patient, enrollment, and ZIP mapping tables), we construct a clean exposure dataset that defines each person-dose observation, applies enrollment and risk-window filters, resolves duplicate and close-dose records, and derives key covariates (age at exposure, season, vaccine brand).

Stage 2 -- Summary and Visualisation (Parts 6a-6b): The analytic dataset is aggregated across eight stratification levels (overall, by season, by brand, by age group, and all pairwise and three-way combinations). For each stratum, we compute the number of vaccinations, unique persons, observed outcomes, person-days at risk, crude incidence rate, and exact Poisson 95% confidence intervals via the chi-squared method.

Downstream Context

This descriptive analysis is the **first step** in a broader evaluation. The seasonal incidence profiles established here will inform:

- Confounder selection and effect modification testing in subsequent regression models.
- The design of self-controlled case series (SCCS) or cohort analyses comparing vaccinated versus unvaccinated risk windows.
- Signal prioritisation for regulatory safety reporting.

By quantifying how myopericarditis rates vary across seasons, brands, and age groups independently, we lay the groundwork for attributing excess risk specifically to vaccination rather than to co-occurring seasonal or demographic factors.

Data: Synthetic administrative claims generated to mirror the structure of real-world insurance data. All person identifiers are simulated. Study period: 2022-2025.

Data Wrangling -- Steps 1-5: Build `exposure_tmp1`

Step 1 -- Construct the base vaccination exposure dataset (subset, merge patient, ZIP enrich, enrollment filter)

Step 2 -- Drop admin concept ID (`724802`) records that co-occur with other concept IDs on the same person-date

Step 3 -- Resolve close-dose pairs (gap <= 3 days): drop same-brand duplicates; drop different-brand pairs

Step 4 -- Derive `age_at_exposure` , `season` , `in_study_window` , `vac_brand`

Step 5 -- Create `after_auth` flag (`1` if vaccination date falls within the brand's authorization window and patient meets the minimum age threshold, else `0`)

```
import pandas as pd
from pathlib import Path

DATA_DIR = Path('dataimages')
```

Load raw datasets

```
DATE_FMT = '%d%b%Y'

def read_csv_with_dates(fname, date_cols):
    df = pd.read_csv(DATA_DIR / fname, dtype=str)
    for col in date_cols:
        df[col] = pd.to_datetime(df[col], format=DATE_FMT, errors='coerce')
    return df

event_raw  = read_csv_with_dates('event.csv',      ['date', 'end_date', 'visit_det_date', 'visit_det_end_date', 'process_date'])
patient    = read_csv_with_dates('patient.csv',    ['birth_date', 'death_date'])
zip3_map   = pd.read_csv(DATA_DIR / 'zip3_mapping.csv', dtype=str)
zip5_map   = pd.read_csv(DATA_DIR / 'zip5_mapping.csv', dtype=str)
enrollment = read_csv_with_dates('enrollment.csv', ['enrl_start_date', 'enrl_end_date'])

print('Rows loaded:')
for name, df in [('event_raw', event_raw), ('patient', patient),
                 ('zip3_map', zip3_map), ('zip5_map', zip5_map), ('enrollment', enrollment)]:
    print(f'  {name:<12} {len(df):>5}')
```

```
Rows loaded:
  event_raw     1000
  patient       1000
  zip3_map      1000
  zip5_map      1000
  enrollment    1000
```

Step 1 -- Subset event.csv to vaccination events within the study window

```
VAX_CODES   = ['cvm24','cvp24','cvn24','cvm23','cvp23','cvn23',
               'cvm22','cvp22','cvn22','cvj22','cvu']
STUDY_START = pd.Timestamp('2022-01-01')
STUDY_END   = pd.Timestamp('2026-01-01')

event_vax = event_raw[event_raw['event'].isin(VAX_CODES)].copy()
event_vax = event_vax[
    (event_vax['date'] >= STUDY_START) &
    (event_vax['date'] <= STUDY_END)
].copy()

print(f'Vaccination events after filter: {len(event_vax):,}')
print(event_vax['event'].value_counts())
```

```
Vaccination events after filter: 939
event
cvu      258
cvm24    132
cvn24    109
cvm23    105
cvp24     93
cvp23     81
cvn23     81
cvp22     23
cvn22     22
cvm22     18
cvj22     17
Name: count, dtype: int64
```

1.2 -- Merge with patient.csv on `person_id`

```
merged = event_vax.merge(patient, on='person_id', how='left')
print(f'Rows after patient merge: {len(merged):,}')
```

```
Rows after patient merge: 939
```

1.3 -- Enrich with ZIP geographic covariates

```
merged['zip']    = merged['zip'].str.strip().str.zfill(5)
merged['zip3']   = merged['zip'].str[:3]
zip5_map['zip5'] = zip5_map['zip5'].str.strip().str.zfill(5)
zip3_map['zip3'] = zip3_map['zip3'].str.strip().str.zfill(3)

merged = merged.merge(zip5_map, left_on='zip', right_on='zip5', how='left')
merged = merged.merge(zip3_map, on='zip3', how='left')
merged.drop(columns=['zip5'], inplace=True)

print(f'Rows after ZIP merges: {len(merged):,}')
print(f'ZIP5 unmatched: {merged["CBSA"].isna().sum():,} | ZIP3 unmatched: {merged["state_abbv"].isna().sum():,}')
```

```
Rows after ZIP merges: 939
ZIP5 unmatched: 0 | ZIP3 unmatched: 0
```

1.4 -- Merge with enrollment.csv and apply date-in-enrollment filter

```
merged = merged.merge(enrollment, on='person_id', how='left')
in_enrollment = (
    (merged['date'] >= merged['enrl_start_date']) &
    (merged['date'] <= merged['enrl_end_date'])
)
exposure_tmp1 = merged[in_enrollment].copy()

print(f'Rows before enrollment filter : {len(merged):,}')
print(f'Rows after  enrollment filter : {len(exposure_tmp1):,}  (exposure_tmp1)')
```

```
Rows before enrollment filter : 939
Rows after  enrollment filter : 939  (exposure_tmp1)
```

Step 2 -- Flag co-occurring admin concept ID (724802)

```
ADMIN_CODE = '724802'
df = exposure_tmp1.copy()

n_concepts_per_person_day = (
    df.groupby(['person_id', 'date'])['concept_id'].transform('nunique')
```

```
)
df['admin_cooccur_flag'] = (
    (df['concept_id'] == ADMIN_CODE) & (n_concepts_per_person_day > 1)
)

n_admin   = (df['concept_id'] == ADMIN_CODE).sum()
n_flagged = df['admin_cooccur_flag'].sum()
print(f'concept_id {ADMIN_CODE} total: {n_admin}  |  flagged (co-occur): {n_flagged}  |  kept (exclusive): {n_admin - n_flagged}')
```

```
concept_id 724802 total: 0  |  flagged (co-occur): 0  |  kept (exclusive): 0
```

```
flagged = df[df['admin_cooccur_flag']][['person_id','date','concept_id','event','visit_det_all_id']].reset_index(drop=True)
if len(flagged) > 0:
    display(flagged)
else:
    print(f'No co-occurring concept_id {ADMIN_CODE} records found. (Expected for synthetic data.)')
```

```
No co-occurring concept_id 724802 records found. (Expected for synthetic data.)
```

```
exposure_tmp1 = df[~df['admin_cooccur_flag']].drop(columns=['admin_cooccur_flag']).copy()
print(f'Rows removed (flagged): {n_flagged:,}  |  Rows retained: {len(exposure_tmp1):,}')
```

```
Rows removed (flagged): 0  |  Rows retained: 939
```

Step 3 -- Resolve close-dose pairs (gap <= 3 days)

Each patient's doses are sorted by date and compared to the immediately preceding dose:

Scenario	Action
Same brand, gap <= 3 days	Drop the later record (`same_brand_dup = True`)
Different brand, gap <= 3 days	Label both records `brand_label = 'multiple'` , then drop them

A `brand` column is derived from the `event` code prefix (`cvp` , `cvm` , `cvn` , `cvj` , `cvu`).
`brand_label` is assigned before dropping so the reason for removal is auditable.

```
# -- extract brand prefix -----------------------------------------------------
# event codes: cvm23 -> 'cvm', cvp24 -> 'cvp', cvu -> 'cvu'
def extract_brand(event_code):
    return 'cvu' if event_code == 'cvu' else event_code[:3]

df = exposure_tmp1.copy()
df['brand'] = df['event'].apply(extract_brand)

# Sort per patient by date so shift() comparisons are chronological
df = df.sort_values(['person_id', 'date']).reset_index(drop=True)

# -- compare each dose to the previous dose of the same patient ---------------
df['_prev_date']  = df.groupby('person_id')['date'].shift(1)
df['_prev_brand'] = df.groupby('person_id')['brand'].shift(1)
df['_gap_days']   = (df['date'] - df['_prev_date']).dt.days

within_3 = df['_gap_days'].notna() & (df['_gap_days'] <= 3)

# -- Rule 1: same brand, <= 3 days -> flag the later dose as a duplicate --------
df['same_brand_dup'] = within_3 & (df['brand'] == df['_prev_brand'])

# -- Rule 2: different brand, <= 3 days -> flag CURRENT and PREVIOUS record -----
# Flag current record (it follows a different-brand dose within 3 days)
is_multi_curr = within_3 & (df['brand'] != df['_prev_brand'])

# Propagate the flag back to the previous record using shift(-1)
df['_next_brand'] = df.groupby('person_id')['brand'].shift(-1)
df['_next_gap']   = df.groupby('person_id')['_gap_days'].shift(-1)
```

```python
is_multi_prev = (
    df['_next_gap'].notna() &
    (df['_next_gap'] <= 3) &
    (df['brand'] != df['_next_brand'])
)

df['multiple_brand'] = is_multi_curr | is_multi_prev

# -- Summarise flags -----------------------------------------------------------
n_same_dup = df['same_brand_dup'].sum()
n_multi    = df['multiple_brand'].sum()
print(f'Same-brand duplicates to drop    : {n_same_dup}')
print(f'Different-brand close pairs      : {n_multi} records will be labelled "multiple"')
```

```
Same-brand duplicates to drop    : 28
Different-brand close pairs      : 8 records will be labelled "multiple"
```

```python
# -- Inspect same-brand duplicates ---------------------------------------------
cols_inspect = ['person_id', 'date', 'event', 'brand', '_prev_date', '_gap_days']

if n_same_dup > 0:
    print(f'--- Same-brand duplicates (later dose dropped) ---')
    display(df[df['same_brand_dup']][cols_inspect].reset_index(drop=True))
else:
    print('No same-brand duplicates within 3 days found.')

print()

# -- Inspect different-brand close pairs ---------------------------------------
if n_multi > 0:
    print(f'--- Different-brand close pairs (labelled "multiple") ---')
    display(
        df[df['multiple_brand']][cols_inspect]
        .sort_values(['person_id', 'date'])
        .reset_index(drop=True)
    )
else:
    print('No different-brand pairs within 3 days found.')
```

```
--- Same-brand duplicates (later dose dropped) ---
```

	person_id	date	event	brand	_prev_date	_gap_days
0	1162230	2023-05-26	cvp23	cvp	2023-05-23	3.0
1	1212267	2023-06-06	cvm23	cvm	2023-06-05	1.0
2	1419610	2023-03-20	cvn23	cvn	2023-03-18	2.0
3	1452421	2024-07-06	cvn24	cvn	2024-07-05	1.0
4	1938483	2023-09-26	cvu	cvu	2023-09-23	3.0
5	2566777	2023-04-28	cvp23	cvp	2023-04-26	2.0
6	2622631	2022-06-06	cvu	cvu	2022-06-05	1.0
7	3500715	2022-11-05	cvj22	cvj	2022-11-04	1.0
8	3566765	2024-06-09	cvp24	cvp	2024-06-06	3.0
9	3762152	2024-02-03	cvm24	cvm	2024-01-31	3.0
10	4337174	2025-01-06	cvu	cvu	2025-01-04	2.0
11	4700025	2024-06-26	cvp24	cvp	2024-06-24	2.0
12	4841005	2024-06-25	cvu	cvu	2024-06-23	2.0
13	4993055	2022-09-08	cvn22	cvn	2022-09-07	1.0
14	5137722	2023-08-12	cvn23	cvn	2023-08-10	2.0
15	5186414	2022-12-06	cvn22	cvn	2022-12-04	2.0
16	5761145	2024-09-03	cvm24	cvm	2024-09-01	2.0
17	6038571	2023-07-30	cvu	cvu	2023-07-27	3.0
18	6492066	2023-12-31	cvp23	cvp	2023-12-30	1.0
19	6573147	2024-10-25	cvu	cvu	2024-10-23	2.0
20	6709938	2022-11-07	cvn22	cvn	2022-11-04	3.0
21	6770619	2023-10-20	cvn23	cvn	2023-10-18	2.0
22	7117837	2024-04-19	cvm24	cvm	2024-04-16	3.0
23	7374122	2022-07-03	cvp22	cvp	2022-06-30	3.0
24	7685149	2023-12-23	cvm23	cvm	2023-12-22	1.0
25	7812810	2024-07-27	cvu	cvu	2024-07-26	1.0
26	8290770	2023-09-05	cvp23	cvp	2023-09-04	1.0
27	8550917	2024-04-05	cvn24	cvn	2024-04-04	1.0

```
--- Different-brand close pairs (labelled "multiple") ---
```

	person_id	date	event	brand	_prev_date	_gap_days
0	2149597	2023-12-19	cvp23	cvp	NaT	NaN
1	2149597	2023-12-20	cvm23	cvm	2023-12-19	1.0
2	2737893	2024-04-13	cvp24	cvp	NaT	NaN
3	2737893	2024-04-16	cvn24	cvn	2024-04-13	3.0
4	7730428	2023-11-21	cvm23	cvm	NaT	NaN
5	7730428	2023-11-24	cvp23	cvp	2023-11-21	3.0
6	8425149	2023-09-24	cvm23	cvm	NaT	NaN
7	8425149	2023-09-25	cvp23	cvp	2023-09-24	1.0

```python
# -- Apply Rule 1: drop same-brand duplicates ----------------------------------
df = df[~df['same_brand_dup']].copy()

# -- Apply Rule 2: label different-brand close pairs, then drop them -----------
df['brand_label'] = df['brand']
df.loc[df['multiple_brand'], 'brand_label'] = 'multiple'

n_multi_dropped = df['multiple_brand'].sum()
df = df[~df['multiple_brand']].copy()

# -- Drop working columns ------------------------------------------------------
working_cols = ['_prev_date', '_prev_brand', '_gap_days', '_next_brand', '_next_gap',
```

```
                'same_brand_dup', 'multiple_brand']
exposure_tmp1 = df.drop(columns=working_cols).copy()

print(f'Rows entering Step 3                    : {len(exposure_tmp1) + n_same_dup + n_multi_dropped:,}')
print(f'Rows dropped  -- same-brand duplicate    : {n_same_dup:,}')
print(f'Rows dropped  -- different-brand close   : {n_multi_dropped:,}')
print(f'Rows retained (exposure_tmp1)           : {len(exposure_tmp1):,}')
print(f'\nbrand_label distribution:')
print(exposure_tmp1['brand_label'].value_counts())
```

```
Rows entering Step 3                    : 939
Rows dropped  -- same-brand duplicate    : 28
Rows dropped  -- different-brand close   : 8
Rows retained (exposure_tmp1)           : 903

brand_label distribution:
brand_label
cvu    251
cvm    247
cvn    203
cvp    186
cvj     16
Name: count, dtype: int64
```

```
print('Step 3 preview (brand and brand_label columns):')
exposure_tmp1[['person_id', 'date', 'event', 'brand', 'brand_label']].head(10)
```

```
Step 3 preview (brand and brand_label columns):
```

	person_id	date	event	brand	brand_label
0	1006810	2023-02-01	cvm23	cvm	cvm
1	1009594	2024-02-14	cvu	cvu	cvu
2	1009594	2024-05-08	cvu	cvu	cvu
3	1028374	2024-04-13	cvm24	cvm	cvm
4	1028374	2024-06-03	cvp24	cvp	cvp
5	1036161	2023-08-16	cvp23	cvp	cvp
6	1036161	2023-12-22	cvp23	cvp	cvp
7	1054447	2025-08-14	cvu	cvu	cvu
8	1059486	2023-08-03	cvn23	cvn	cvn
9	1059486	2023-09-03	cvm23	cvm	cvm

Step 4 -- Derive analytical variables

Variable	Definition
`age_at_exposure`	Completed years between `birth_date` and vaccination `date` (floor division of day difference by 365.25)
`season`	COVID19-year season: `22_23` (01Jul2022-30Jun2023), `23_24` (01Jul2023-30Jun2024), `24_25` (01Jul2024-30Jun2025); `None` if outside all windows
`in_study_window`	`1` if `01Jul2022 <= date <= 30Jun2025`, else `0`
`vac_brand`	`{brand_name}_{season}` for all brands including `cvu` (e.g. `moderna_23_24`, `Unknown_brand_23_24`); `NaN` for records outside defined season windows

```
df = exposure_tmp1.copy()

# -- 1. Age at exposure (completed years) -------------------------------------
df['age_at_exposure'] = ((df['date'] - df['birth_date']).dt.days / 365.25).astype(int)

# -- 2. Season (Jul-Jun COVID19-year boundaries) ----------------------------------
df['season'] = None
df.loc[(df['date'] >= '2022-07-01') & (df['date'] <= '2023-06-30'), 'season'] = '22_23'
df.loc[(df['date'] >= '2023-07-01') & (df['date'] <= '2024-06-30'), 'season'] = '23_24'
```

```python
df.loc[(df['date'] >= '2024-07-01') & (df['date'] <= '2025-06-30'), 'season'] = '24_25'

# -- 3. in_study_window flag --------------------------------------------------
df['in_study_window'] = (
    (df['date'] >= pd.Timestamp('2022-07-01')) &
    (df['date'] <= pd.Timestamp('2025-06-30'))
).astype(int)

print('age_at_exposure:')
print(f"  Range : {df['age_at_exposure'].min()} - {df['age_at_exposure'].max()} years")
print(f"  Mean  : {df['age_at_exposure'].mean():.1f} years")
print()
print('season distribution:')
print(df['season'].value_counts(dropna=False))
print()
print('in_study_window distribution:')
print(df['in_study_window'].value_counts())
```

```
age_at_exposure:
  Range : 11 - 28 years
  Mean  : 19.5 years

season distribution:
season
23_24    386
24_25    267
22_23    212
None      38
Name: count, dtype: int64

in_study_window distribution:
in_study_window
1    865
0     38
Name: count, dtype: int64
```

```python
# -- 4. vac_brand -- brand x season label -------------------------------------
BRAND_FULL = {
    'cvp': 'pfizer',
    'cvm': 'moderna',
    'cvn': 'novavax',
    'cvj': 'janssen',
    'cvu': 'Unknown_brand',   # season suffix appended just like named brands
}

df['_brand_full'] = df['brand'].map(BRAND_FULL)

# Combine brand name and season; NaN propagates where either is NaN
df['vac_brand'] = df['_brand_full'] + '_' + df['season']

df.drop(columns=['_brand_full'], inplace=True)

print('vac_brand distribution:')
print(df['vac_brand'].value_counts(dropna=False))
```

```
vac_brand distribution:
vac_brand
moderna_23_24          118
Unknown_brand_24_25    101
pfizer_23_24            91
novavax_23_24           90
Unknown_brand_23_24     87
moderna_24_25           67
moderna_22_23           59
novavax_24_25           58
novavax_22_23           53
pfizer_22_23            46
pfizer_24_25            41
Unknown_brand_22_23     41
NaN                     38
janssen_22_23           13
Name: count, dtype: int64
```

```python
exposure_tmp1 = df.copy()

print('--- Step 4 summary ---')
print(f'Shape: {exposure_tmp1.shape}')
print()
new_cols = ['age_at_exposure', 'season', 'in_study_window', 'vac_brand']
print('Sample of new columns:')
print(exposure_tmp1[['person_id', 'date', 'brand'] + new_cols].head(12).to_string(index=False))
```

```
--- Step 4 summary ---
Shape: (903, 37)

Sample of new columns:
person_id       date brand  age_at_exposure season  in_study_window           vac_brand
  1006810 2023-02-01   cvm               14  22_23                1       moderna_22_23
  1009594 2024-02-14   cvu               13  23_24                1 Unknown_brand_23_24
  1009594 2024-05-08   cvu               13  23_24                1 Unknown_brand_23_24
  1028374 2024-04-13   cvm               23  23_24                1       moderna_23_24
  1028374 2024-06-03   cvp               23  23_24                1        pfizer_23_24
  1036161 2023-08-16   cvp               16  23_24                1        pfizer_23_24
  1036161 2023-12-22   cvp               17  23_24                1        pfizer_23_24
  1054447 2025-08-14   cvu               23   None                0                 NaN
  1059486 2023-08-03   cvn               16  23_24                1       novavax_23_24
  1059486 2023-09-03   cvm               16  23_24                1       moderna_23_24
  1093026 2023-08-04   cvp               19  23_24                1        pfizer_23_24
  1093026 2023-09-22   cvp               19  23_24                1        pfizer_23_24
```

Step 5 -- After-authorization flag (`after_auth`)

`after_auth = 1` when **all three** conditions are met; `0` otherwise:

1. `vac_brand` matches a defined authorization entry
2. `age_at_exposure` meets the brand's minimum age threshold
3. `date` falls within the brand's authorization window

Brand group	Min age	Authorization window
`moderna_22_23`, `pfizer_22_23`, `novavax_22_23`, `Unknown_brand_22_23`	>= 12	01Aug2022 - 30Jun2023
`moderna_23_24`, `pfizer_23_24`, `novavax_23_24`, `Unknown_brand_23_24`	>= 12	01Aug2023 - 30Jun2024
`moderna_24_25`, `pfizer_24_25`, `novavax_24_25`, `Unknown_brand_24_25`	>= 12	01Aug2024 - 30Jun2025
`janssen_22_23`	>= **18**	01Aug2022 - 30Jun2023

Records with `vac_brand = NaN` (outside all season windows) receive `after_auth = 0`.

```python
# Authorization conditions per vac_brand: (min_age, auth_start, auth_end)
AUTH_CONDITIONS = {
    'moderna_22_23':       (12, '2022-08-01', '2023-06-30'),
    'moderna_23_24':       (12, '2023-08-01', '2024-06-30'),
    'moderna_24_25':       (12, '2024-08-01', '2025-06-30'),
    'pfizer_22_23':        (12, '2022-08-01', '2023-06-30'),
    'pfizer_23_24':        (12, '2023-08-01', '2024-06-30'),
    'pfizer_24_25':        (12, '2024-08-01', '2025-06-30'),
    'novavax_22_23':       (12, '2022-08-01', '2023-06-30'),
    'novavax_23_24':       (12, '2023-08-01', '2024-06-30'),
    'novavax_24_25':       (12, '2024-08-01', '2025-06-30'),
    'janssen_22_23':       (18, '2022-08-01', '2023-06-30'),   # Janssen: age >= 18
    'Unknown_brand_22_23': (12, '2022-08-01', '2023-06-30'),
    'Unknown_brand_23_24': (12, '2023-08-01', '2024-06-30'),
    'Unknown_brand_24_25': (12, '2024-08-01', '2025-06-30'),
}

df = exposure_tmp1.copy()
df['after_auth'] = 0

for vac_brand, (min_age, auth_start, auth_end) in AUTH_CONDITIONS.items():
    mask = (
        (df['vac_brand'] == vac_brand) &
        (df['age_at_exposure'] >= min_age) &
        (df['date'] >= pd.Timestamp(auth_start)) &
        (df['date'] <= pd.Timestamp(auth_end))
    )
    df.loc[mask, 'after_auth'] = 1

print('after_auth distribution:')
print(df['after_auth'].value_counts())
print()
print('after_auth by vac_brand (1 = authorised, 0 = not):')
summary = (
    df.groupby('vac_brand', dropna=False)['after_auth']
    .agg(after_auth_1='sum', total='count')
    .assign(not_auth=lambda x: x['total'] - x['after_auth_1'])
)
print(summary.to_string())
```

```
after_auth distribution:
after_auth
1    791
0    112
Name: count, dtype: int64

after_auth by vac_brand (1 = authorised, 0 = not):
                     after_auth_1  total  not_auth
vac_brand
Unknown_brand_22_23            41     41         0
Unknown_brand_23_24            78     87         9
Unknown_brand_24_25            93    101         8
janssen_22_23                  11     13         2
moderna_22_23                  56     59         3
moderna_23_24                 111    118         7
moderna_24_25                  52     67        15
novavax_22_23                  52     53         1
novavax_23_24                  84     90         6
novavax_24_25                  48     58        10
pfizer_22_23                   45     46         1
pfizer_23_24                   87     91         4
pfizer_24_25                   33     41         8
NaN                             0     38        38
```

```python
exposure_tmp1 = df.copy()

print(f'Shape: {exposure_tmp1.shape}')
print()
print('Sample (after_auth column):')
```

```
print(
    exposure_tmp1[['person_id', 'date', 'age_at_exposure', 'vac_brand', 'after_auth']]
    .head(14)
    .to_string(index=False)
)
```

```
Shape: (903, 38)

Sample (after_auth column):
person_id       date  age_at_exposure           vac_brand  after_auth
  1006810 2023-02-01               14       moderna_22_23           1
  1009594 2024-02-14               13 Unknown_brand_23_24           1
  1009594 2024-05-08               13 Unknown_brand_23_24           1
  1028374 2024-04-13               23       moderna_23_24           1
  1028374 2024-06-03               23        pfizer_23_24           1
  1036161 2023-08-16               16        pfizer_23_24           1
  1036161 2023-12-22               17        pfizer_23_24           1
  1054447 2025-08-14               23                 NaN           0
  1059486 2023-08-03               16       novavax_23_24           1
  1059486 2023-09-03               16       moderna_23_24           1
  1093026 2023-08-04               19        pfizer_23_24           1
  1093026 2023-09-22               19        pfizer_23_24           1
  1098919 2024-09-06               23 Unknown_brand_24_25           1
  1109031 2024-06-24               17        pfizer_23_24           1
```

Attrition Table -- Exposure Dataset Construction (Part 1)

Records excluded at each step from the raw event file through to `exposure_enriched.csv`.

```
# -- Attrition Table: Exposure Dataset Construction (Part 1) ---------------------

# Re-read source files to reconstruct step-level record counts
_ev  = pd.read_csv(DATA_DIR / 'event.csv')
_ev['date'] = pd.to_datetime(_ev['date'], format='%d%b%Y', errors='coerce')
_ev['concept_id'] = _ev['concept_id'].fillna('').astype(str)

_pt  = pd.read_csv(DATA_DIR / 'patient.csv')
_en  = pd.read_csv(DATA_DIR / 'enrollment.csv')
for _c in ['enrl_start_date', 'enrl_end_date']:
    _en[_c] = pd.to_datetime(_en[_c], format='%d%b%Y', errors='coerce')

# Step 0 -- raw event records
_n0 = len(_ev)

# Step 1.1 -- subset to vaccination event codes
_vax = _ev[_ev['event'].isin(VAX_CODES)].copy()
_n1 = len(_vax)

# Step 1.2 -- inner merge with patient file (must have matching person_id)
_vax_pt = _vax.merge(_pt[['person_id']], on='person_id', how='inner')
_n2 = len(_vax_pt)

# Step 1.3 -- ZIP geographic enrichment (left join -- no row loss)
_n3 = _n2

# Step 1.4 -- restrict to enrollment window (date within enrl_start/end)
_vax_enrl = _vax_pt.merge(_en[['person_id', 'enrl_start_date', 'enrl_end_date']],
                           on='person_id', how='left')
_vax_enrl_in = _vax_enrl[
    _vax_enrl['date'].notna() &
    (_vax_enrl['date'] >= _vax_enrl['enrl_start_date']) &
    (_vax_enrl['date'] <= _vax_enrl['enrl_end_date'])
].copy()
_n4 = len(_vax_enrl_in)
```

```
# Step 2 -- remove admin concept co-occurrence (concept_id == ADMIN_CODE on same person+date)
_admin_pairs = set(zip(
    _ev.loc[_ev['concept_id'] == ADMIN_CODE, 'person_id'].astype(str),
    _ev.loc[_ev['concept_id'] == ADMIN_CODE, 'date'].astype(str)
))
_keep_mask = ~_vax_enrl_in.apply(
    lambda r: (str(r['person_id']), str(r['date'])) in _admin_pairs, axis=1
)
_n5 = int(_keep_mask.sum())

# Step 3 -- close-dose pair resolution (final count = current exposure_tmp1)
# Steps 4 & 5 only add columns; row count unchanged.
_n6 = len(exposure_tmp1)

# -- Build attrition DataFrame -------------------------------------------------------
_rows = [
    ('All records in event.csv',                                  _n0, '--'),
    ('Step 1.1  Subset to vaccination event codes',               _n1, _n0 - _n1),
    ('Step 1.2  Inner merge with patient file',                   _n2, _n1 - _n2),
    ('Step 1.3  ZIP geographic enrichment (left join)',           _n3, 0),
    ('Step 1.4  Restrict to enrollment window',                  _n4, _n3 - _n4),
    ('Step 2    Remove admin concept co-occurrence (724802)',     _n5, _n4 - _n5),
    ('Step 3    Resolve close-dose pairs (<=3 days)',             _n6, _n5 - _n6),
    ('Step 4    Derive analytical variables (columns added only)',  _n6, 0),
    ('Step 5    Apply after-authorization flag (column added only)',_n6, 0),
]

_attrition = pd.DataFrame(_rows, columns=['Step', 'N Remaining', 'N Excluded'])
_attrition['N Excluded'] = _attrition['N Excluded'].apply(
    lambda x: x if x == '--' else (0 if int(x) == 0 else f'-{int(x)}')
)
_attrition.index = range(1, len(_attrition) + 1)
_attrition.index.name = '#'

print('Attrition Table -- Exposure Dataset Construction (Part 1)')
print('=' * 75)
print(_attrition.to_string())
print()
print(f"  -> Final exposure_enriched.csv : {_n6:>4} records | "
      f"{exposure_tmp1['person_id'].nunique()} unique persons")
```

```
Attrition Table -- Exposure Dataset Construction (Part 1)
===========================================================================
                                                           Step  N Remaining N Excluded
#
1                                      All records in event.csv         1000         --
2                   Step 1.1  Subset to vaccination event codes          939        -61
3                       Step 1.2  Inner merge with patient file          939          0
4               Step 1.3  ZIP geographic enrichment (left join)          939          0
5                       Step 1.4  Restrict to enrollment window          939          0
6         Step 2    Remove admin concept co-occurrence (724802)          939          0
7                   Step 3    Resolve close-dose pairs (<=3 days)        903        -36
8    Step 4    Derive analytical variables (columns added only)          903          0
9  Step 5    Apply after-authorization flag (column added only)          903          0

  -> Final exposure_enriched.csv :  903 records | 539 unique persons
```

Save final dataset -- `exposure_enriched.csv`

```
out_path = DATA_DIR / 'exposure_enriched.csv'
try:
    exposure_tmp1.to_csv(out_path, index=False)
    print(f'Saved {len(exposure_tmp1):,} rows  x  {exposure_tmp1.shape[1]} columns  ->  {out_path}')
except PermissionError:
    print(f'Warning: {out_path.name} is locked by another application -- skipping save (in-memory copy is current).')
print(f'\nFinal columns:')
print(list(exposure_tmp1.columns))
```

```
Saved 903 rows  x  38 columns  ->  dataimages\exposure_enriched.csv

Final columns:
['person_id', 'visit_det_all_id', 'event', 'concept_id', 'source_concept_id', 'date', 'end_date', 'event_num', 'dqc_posit_ion', 'rv_visit_details', 'visit_det_all_concept_pt_id', 'visit_det_date', 'visit_det_end_date',
'visit_det_aim', 'setting', 'facility', 'process_date', 'sex', 'birth_date', 'death_date', 'zip', 'zip3', 'CBSA', 'urban', 'state_fips', 'state_name', 'state_abbv', 'omb_region_label', 'census_region_label', 'enrl_start_date',
'enrl_end_date', 'brand', 'brand_label', 'age_at_exposure', 'season', 'in_study_window', 'vac_brand', 'after_auth']
```

Part 2 -- Outcome Data Cleaning

Outcome Step 1 -- Import `event.csv`, subset to myocarditis events within the study window

Filters applied:

- `event == 'myo'`
- `01Jul2022 <= date <= 30Jun2025`

Season boundaries (same Jul-Jun convention as exposure data):

Season	Period
`22_23`	01 Jul 2022 - 30 Jun 2023
`23_24`	01 Jul 2023 - 30 Jun 2024
`24_25`	01 Jul 2024 - 30 Jun 2025

```
# -- Outcome Step 1: load, subset, and create season --------------------------
MYO_START = pd.Timestamp('2022-07-01')
MYO_END   = pd.Timestamp('2025-06-30')

event_outcome = read_csv_with_dates(
    'event.csv',
    ['date', 'end_date', 'visit_det_date', 'visit_det_end_date', 'process_date']
)

myo_df = event_outcome[
    (event_outcome['event'] == 'myo') &
    (event_outcome['date'] >= MYO_START) &
    (event_outcome['date'] <= MYO_END)
].copy()

# Season variable (same Jul-Jun boundaries as exposure data)
myo_df['season'] = None
myo_df.loc[(myo_df['date'] >= '2022-07-01') & (myo_df['date'] <= '2023-06-30'), 'season'] = '22_23'
myo_df.loc[(myo_df['date'] >= '2023-07-01') & (myo_df['date'] <= '2024-06-30'), 'season'] = '23_24'
myo_df.loc[(myo_df['date'] >= '2024-07-01') & (myo_df['date'] <= '2025-06-30'), 'season'] = '24_25'

print(f'Total myo events in study window (01Jul2022-30Jun2025): {len(myo_df):,}')
print(f'Unique patients                                        : {myo_df["person_id"].nunique():,}')
print()
print('Season distribution:')
print(myo_df['season'].value_counts(dropna=False))
print()
print('Date range:')
print(f"  Min: {myo_df['date'].min().date()}  |  Max: {myo_df['date'].max().date()}")
```

```
Total myo events in study window (01Jul2022-30Jun2025): 58
Unique patients                                        : 58

Season distribution:
season
23_24    27
24_25    21
22_23    10
Name: count, dtype: int64

Date range:
  Min: 2022-09-30  |  Max: 2025-06-14
```

Outcome Step 2 -- Create `myo_event` flag

`myo_event = 1` for every myocarditis record retained in the outcome dataset (all rows satisfy the condition by construction after the Step 1 filter).
Where a patient has more than one myocarditis event within the same season, only the **first** event (earliest date) is kept as the index case.

```python
# -- Step 2: myo_event flag ----------------------------------------------------
# Every record in myo_df is a myo event by construction -> flag = 1 for all
# Deduplicate to the FIRST event per person per season (index case only)
myo_df = (
    myo_df.sort_values(['person_id', 'season', 'date'])
          .drop_duplicates(subset=['person_id', 'season'], keep='first')
          .copy()
)

myo_df['myo_event'] = 1

print(f'Myo records (first event per person-season): {len(myo_df):,}')
print(f'Unique patients                            : {myo_df["person_id"].nunique():,}')
print()
print('myo_event by season:')
print(myo_df.groupby('season')['myo_event'].sum())
print()
print('Preview:')
print(myo_df[['person_id', 'date', 'season', 'myo_event']].head(10).to_string(index=False))
```

```
Myo records (first event per person-season): 58
Unique patients                            : 58

myo_event by season:
season
22_23    10
23_24    27
24_25    21
Name: myo_event, dtype: int64

Preview:
person_id       date season  myo_event
  1093026 2023-10-08  23_24          1
  1109031 2024-09-17  24_25          1
  1192619 2024-08-11  24_25          1
  1481506 2024-10-07  24_25          1
  1533224 2024-09-06  24_25          1
  1666754 2025-01-27  24_25          1
  1841248 2023-11-18  23_24          1
  1959074 2024-03-16  23_24          1
  1971823 2023-06-01  22_23          1
  2164686 2024-01-16  23_24          1
```

Outcome Step 3 -- 365-day washout between myocarditis events

Condition: For a given individual, consecutive myocarditis events must be **>= 365 days apart** -- both within the same season and across seasons.

- Events are sorted chronologically per patient.
- The **first** event is always retained.
- Each subsequent event is retained only if it is >= 365 days from the last retained event; otherwise it is dropped.
- This prevents a single prolonged or recurrent episode from being counted multiple times across season boundaries.

```
# Apply 365-day washout using an explicit per-patient loop (avoids pandas groupby.apply warnings)
myo_flagged = myo_df.copy()
myo_flagged['keep_365d'] = False

for pid, grp in myo_df.groupby('person_id'):
    grp = grp.sort_values('date')
    keep_flags = [True]
    last_kept = grp.iloc[0]['date']
    for i in range(1, len(grp)):
        gap = (grp.iloc[i]['date'] - last_kept).days
        if gap >= 365:
            keep_flags.append(True)
            last_kept = grp.iloc[i]['date']
        else:
            keep_flags.append(False)
    myo_flagged.loc[grp.index, 'keep_365d'] = keep_flags

n_before  = len(myo_flagged)
n_dropped = (~myo_flagged['keep_365d']).sum()
n_after   = myo_flagged['keep_365d'].sum()

print(f'Myo events before 365-day washout : {n_before:,}')
print(f'Dropped (gap < 365 days)          : {n_dropped:,}')
print(f'Retained                          : {n_after:,}')

dropped = myo_flagged[~myo_flagged['keep_365d']][['person_id', 'date', 'season', 'myo_event']]
if len(dropped) > 0:
    print()
    print('Dropped records (too close to a prior myo event):')
    print(dropped.to_string(index=False))
else:
    print()
    print('No records dropped -- all myo events are >= 365 days apart for every individual.')

myo_df = myo_flagged[myo_flagged['keep_365d']].drop(columns=['keep_365d']).copy()

print()
print('Final myo_event counts by season:')
print(myo_df.groupby('season')['myo_event'].sum())
```

```
Myo events before 365-day washout : 58
Dropped (gap < 365 days)          : 0
Retained                          : 58

No records dropped -- all myo events are >= 365 days apart for every individual.

Final myo_event counts by season:
season
22_23    10
23_24    27
24_25    21
Name: myo_event, dtype: int64
```

```
out_path = DATA_DIR / 'myo_events.csv'
try:
    myo_df.to_csv(out_path, index=False)
    print(f'Saved {len(myo_df):,} rows  x  {myo_df.shape[1]} columns  ->  {out_path}')
except PermissionError:
    print(f'Warning: {out_path.name} is locked by another application -- skipping save (in-memory copy is current).')
print(f'\nColumns: {list(myo_df.columns)}')
```

```
Saved 58 rows  x  19 columns  ->  dataimages\myo_events.csv

Columns: ['person_id', 'visit_det_all_id', 'event', 'concept_id', 'source_concept_id', 'date', 'end_date', 'event_num', 'dqc_posit_ion', 'rv_visit_details', 'visit_det_all_concept_pt_id', 'visit_det_date', 'visit_det_end_date', 'visit_det_aim', 'setting', 'facility', 'process_date', 'season', 'myo_event']
```

Part 3 -- Merge Exposure and Outcome Data

Standard approach: for each individual in a given season, a myocarditis event is linked to their vaccination record only if `myo_date > first_vax_date` (i.e., myocarditis occurred *after* the first vaccination in that season).

Steps:

1. Derive `first_vax_date` -- the earliest vaccination date per `(person_id, season)` in `exposure_enriched`
2. Left-join `myo_events` on `(person_id, season)` to bring in `myo_date`
3. Flag `myo_after_first_vax = 1` where `myo_date > first_vax_date`; else `0`
4. Merge the flags back onto all vaccination records in `exposure_enriched`

```python
# -- Load both clean datasets --------------------------------------------------
exposure = pd.read_csv(DATA_DIR / 'exposure_enriched.csv')
myo      = pd.read_csv(DATA_DIR / 'myo_events.csv')

# Explicit date conversion (avoids dtype=str + parse_dates interaction issues)
for col in ['date', 'end_date', 'birth_date', 'death_date',
            'visit_det_date', 'visit_det_end_date', 'process_date',
            'enrl_start_date', 'enrl_end_date']:
    if col in exposure.columns:
        exposure[col] = pd.to_datetime(exposure[col], errors='coerce')

for col in ['date', 'end_date', 'visit_det_date', 'visit_det_end_date', 'process_date']:
    if col in myo.columns:
        myo[col] = pd.to_datetime(myo[col], errors='coerce')

exposure['person_id'] = exposure['person_id'].astype(str)
myo['person_id']      = myo['person_id'].astype(str)

print(f'exposure_enriched : {exposure.shape}')
print(f'myo_events        : {myo.shape}')

# -- Step 1: First vaccination date per person x season -----------------------
first_vax = (
    exposure
    .groupby(['person_id', 'season'])['date']
    .min()
    .reset_index()
    .rename(columns={'date': 'first_vax_date'})
)

# -- Step 2: Myo events per person x season -----------------------------------
myo_link = (
    myo[['person_id', 'season', 'date', 'myo_event']]
    .rename(columns={'date': 'myo_date'})
)

# Left join on person_id + season -- keep all vaccinated person-seasons
person_season = first_vax.merge(myo_link, on=['person_id', 'season'], how='left')

# -- Step 3: Flag myo only if it occurred AFTER the first vaccination ----------
person_season['myo_after_first_vax'] = (
    person_season['myo_date'].notna() &
    (person_season['myo_date'] > person_season['first_vax_date'])
).astype('int64')
```

```python
n_qualifying = person_season['myo_after_first_vax'].sum()
n_excluded   = (person_season['myo_date'].notna() & (person_season['myo_after_first_vax'] == 0)).sum()

print(f'\nPerson-season pairs with a myo event                  : {person_season["myo_date"].notna().sum():,}')
print(f'  myo_date > first_vax_date  (myo_after_first_vax=1)  : {n_qualifying:,}')
print(f'  myo_date <= first_vax_date  (excluded)               : {n_excluded:,}')

# -- Step 4: Merge flags back onto all vaccination records ---------------------
merged = exposure.merge(
    person_season[['person_id', 'season', 'first_vax_date', 'myo_date', 'myo_after_first_vax']],
    on=['person_id', 'season'],
    how='left'
)
merged['myo_after_first_vax'] = merged['myo_after_first_vax'].fillna(0).astype('int64')

print(f'\nMerged dataset shape : {merged.shape}')
print(f'\nmyo_after_first_vax distribution:')
print(merged['myo_after_first_vax'].value_counts())
print()
print('myo_after_first_vax by season:')
print(merged.groupby('season')['myo_after_first_vax'].sum())
print()

# Preview -- use .dt.strftime directly since columns are already datetime
cols = ['person_id', 'date', 'season', 'first_vax_date', 'myo_date', 'myo_after_first_vax']
preview = (
    merged[merged['myo_after_first_vax'] == 1][cols]
    .drop_duplicates('person_id')
    .head(8)
    .copy()
)
preview['date']           = preview['date'].dt.strftime('%d%b%Y')
preview['first_vax_date'] = preview['first_vax_date'].dt.strftime('%d%b%Y')
preview['myo_date']       = preview['myo_date'].dt.strftime('%d%b%Y')
print('Preview -- confirmed cases (myo_date > first_vax_date):')
print(preview.to_string(index=False))
```

```
exposure_enriched : (903, 38)
myo_events        : (58, 19)

Person-season pairs with a myo event                  : 57
  myo_date > first_vax_date  (myo_after_first_vax=1)  : 57
  myo_date <= first_vax_date  (excluded)               : 0

Merged dataset shape : (903, 41)

myo_after_first_vax distribution:
myo_after_first_vax
0    822
1     81
Name: count, dtype: int64

myo_after_first_vax by season:
season
22_23    16
23_24    40
24_25    25
Name: myo_after_first_vax, dtype: int64

Preview -- confirmed cases (myo_date > first_vax_date):
person_id      date season first_vax_date  myo_date  myo_after_first_vax
  1093026 04Aug2023  23_24      04Aug2023 08Oct2023                    1
  1109031 27Aug2024  24_25      27Aug2024 17Sep2024                    1
  1192619 19Jul2024  24_25      19Jul2024 11Aug2024                    1
  1481506 05Oct2024  24_25      05Oct2024 07Oct2024                    1
  1533224 17Aug2024  24_25      17Aug2024 06Sep2024                    1
  1666754 06Dec2024  24_25      06Dec2024 27Jan2025                    1
  1841248 29Oct2023  23_24      29Oct2023 18Nov2023                    1
  1959074 14Aug2023  23_24      14Aug2023 16Mar2024                    1
```

Part 4 -- Age Categorisation and Inclusion Restrictions

Age category (`age_cat`)

Category	Condition
`12_17`	`age_at_exposure >= 12` and `< 18`
`18_24`	`age_at_exposure >= 18` and `< 25`
`other`	all else

Inclusion criteria (applied sequentially)

1. `after_auth = 1` -- vaccination within brand-specific authorization window, meeting minimum age
2. `in_study_window = 1` -- vaccination date within 01Jul2022-30Jun2025
3. `age_cat` in `{12_17, 18_24}` -- exclude records outside the target age range
4. Remove `Unknown_brand` -- exclude records where `vac_brand` contains `Unknown_brand`

```
df = merged.copy()
n_start = len(df)

# -- Age category ----------------------------------------------------------------
age = df['age_at_exposure'].astype(int)
df['age_cat'] = 'other'
df.loc[(age >= 12) & (age < 18), 'age_cat'] = '12_17'
df.loc[(age >= 18) & (age < 25), 'age_cat'] = '18_24'

print('age_cat distribution (before restrictions):')
print(df['age_cat'].value_counts(dropna=False))
```

```python
print()

# -- Apply inclusion criteria -------------------------------------------------
steps = [
    ('Start',                  df.copy()),
]

# 1. after_auth = 1
df = df[df['after_auth'] == 1].copy()
steps.append(('after_auth = 1', df.copy()))

# 2. in_study_window = 1
df = df[df['in_study_window'] == 1].copy()
steps.append(('in_study_window = 1', df.copy()))

# 3. age_cat in target groups
df = df[df['age_cat'].isin(['12_17', '18_24'])].copy()
steps.append(('age_cat in {12_17, 18_24}', df.copy()))

# 4. Remove Unknown_brand
df = df[~df['vac_brand'].str.contains('Unknown_brand', na=True)].copy()
steps.append(('Remove Unknown_brand', df.copy()))

# -- Attrition table ----------------------------------------------------------
print(f'{"Step":<30}  {"N records":>10}  {"Removed":>9}')
print('-' * 55)
prev_n = n_start
for label, frame in steps:
    n = len(frame)
    removed = prev_n - n if label != 'Start' else 0
    print(f'{label:<30}  {n:>10,}  {removed:>9,}')
    prev_n = n

analytic = df.copy()
print()
print(f'Analytic dataset shape : {analytic.shape}')
print()
print('age_cat:')
print(analytic['age_cat'].value_counts())
print()
print('vac_brand:')
print(analytic['vac_brand'].value_counts(dropna=False))
print()
print('myo_after_first_vax:')
print(analytic['myo_after_first_vax'].value_counts())
```

```
age_cat distribution (before restrictions):
age_cat
18_24    462
12_17    312
other    129
Name: count, dtype: int64

Step                              N records    Removed
-------------------------------------------------------
Start                                   903          0
after_auth = 1                          791        112
in_study_window = 1                     791          0
age_cat in {12_17, 18_24}               680        111
Remove Unknown_brand                    501        179

Analytic dataset shape : (501, 42)

age_cat:
age_cat
18_24    312
12_17    189
Name: count, dtype: int64

vac_brand:
vac_brand
moderna_23_24    97
pfizer_23_24     74
novavax_23_24    72
moderna_22_23    52
novavax_22_23    44
pfizer_22_23     43
moderna_24_25    41
novavax_24_25    40
pfizer_24_25     27
janssen_22_23    11
Name: count, dtype: int64

myo_after_first_vax:
myo_after_first_vax
0    452
1     49
Name: count, dtype: int64
```

```
out_path = DATA_DIR / 'analytic_dataset.csv'
try:
    analytic.to_csv(out_path, index=False)
    print(f'Saved {len(analytic):,} rows  x  {analytic.shape[1]} columns  ->  {out_path}')
except PermissionError:
    print(f'Warning: {out_path.name} is locked by another application -- skipping save.')
print(f'\nColumns: {list(analytic.columns)}')
```

```
Saved 501 rows  x  42 columns  ->  dataimages\analytic_dataset.csv

Columns: ['person_id', 'visit_det_all_id', 'event', 'concept_id', 'source_concept_id', 'date', 'end_date', 'event_num', 'dqc_posit_ion', 'rv_visit_details', 'visit_det_all_concept_pt_id', 'visit_det_date', 'visit_det_end_date', 'visit_det_aim', 'setting', 'facility', 'process_date', 'sex', 'birth_date', 'death_date', 'zip', 'zip3', 'CBSA', 'urban', 'state_fips', 'state_name', 'state_abbv', 'omb_region_label', 'census_region_label', 'enrl_start_date', 'enrl_end_date', 'brand', 'brand_label', 'age_at_exposure', 'season', 'in_study_window', 'vac_brand', 'after_auth', 'first_vax_date', 'myo_date', 'myo_after_first_vax', 'age_cat']
```

Part 5 -- Risk Windows and Follow-up Time

The analysis is anchored at the **first vaccination date per person x season** (`first_vax_date`).

Risk window

Parameter	Value
Risk window start	`first_vax_date + 0 days` (`risk_start_dt`)
Risk window end (uncensored)	`first_vax_date + 28 days`
Max follow-up	28 days

Censoring rules (minimum date wins)

Priority	Event	`censor_reason`
1	Disenrollment -- `enrl_end_date`	`disenrollment`
2	Death -- `death_date`	`death`
3	Season study end (30 Jun of each COVID19-year)	`study_end`
4	Risk window end -- `first_vax_date + 28 d`	`observation_end`
5	Day before next vaccination -- `next_vax_date - 1 d`	`next_dose`

Standard approach for next-dose censoring: identify the earliest vaccination date for that person that is strictly after `first_vax_date` (across all seasons). Censor at `next_vax_date - 1 day` so the observation window does not overlap with the risk period of the subsequent dose.

Derived variables

Variable	Formula
`risk_end_dt`	`min(enrl_end_date, death_date, season_end, first_vax_date+28, next_vax_date-1)`
`risk_length`	`(risk_end_dt - risk_start_dt).days + 1` (inclusive interval)
`obs_risk_length`	equals `risk_length` -- person-days contributed in the risk window
`outcome_in_risk`	`1` if `risk_start_dt <= myo_date <= risk_end_dt` , else `0`

```
RISK_WIN_START_DAYS = 0
RISK_WIN_END_DAYS   = 28

SEASON_END_DT = {
    '22_23': pd.Timestamp('2023-06-30'),
    '23_24': pd.Timestamp('2024-06-30'),
    '24_25': pd.Timestamp('2025-06-30'),
}

# -- Step 1: Collapse to first dose per person x season -----------------------
keep_cols = [
    'person_id', 'season', 'first_vax_date', 'myo_date', 'myo_after_first_vax',
    'enrl_end_date', 'death_date', 'age_at_exposure', 'age_cat', 'sex',
    'vac_brand', 'brand', 'brand_label', 'after_auth', 'in_study_window',
    'state_abbv', 'urban', 'census_region_label', 'CBSA',
]
keep_cols = [c for c in keep_cols if c in analytic.columns]

person_season = (
    analytic
    .sort_values(['person_id', 'season', 'date'])
    .drop_duplicates(subset=['person_id', 'season'], keep='first')
    [keep_cols]
    .copy()
    .reset_index(drop=True)
)
print(f'Person-season records : {len(person_season):,}')
print(f'Unique persons        : {person_season["person_id"].nunique():,}')

# -- Step 2: Next vaccination date per person (standard approach) --------------
```

```python
# For each person, find the earliest vaccination date STRICTLY AFTER first_vax_date
# (any season). Censor observation at next_vax_date - 1 day.
all_vax = (
    analytic[['person_id', 'date']]
    .drop_duplicates()
    .rename(columns={'date': 'vax_date'})
)
candidates = (
    person_season[['person_id', 'season', 'first_vax_date']]
    .merge(all_vax, on='person_id', how='left')
)
candidates = candidates[candidates['vax_date'] > candidates['first_vax_date']]
next_dose = (
    candidates
    .groupby(['person_id', 'season'])['vax_date']
    .min()
    .reset_index()
    .rename(columns={'vax_date': 'next_vax_date'})
)
person_season = person_season.merge(next_dose, on=['person_id', 'season'], how='left')

# -- Step 3: Risk window boundaries ------------------------------------------
person_season['risk_start_dt']       = person_season['first_vax_date'] + pd.Timedelta(days=RISK_WIN_START_DAYS)
person_season['risk_end_uncensored'] = person_season['first_vax_date'] + pd.Timedelta(days=RISK_WIN_END_DAYS)
person_season['season_end_dt']       = person_season['season'].map(SEASON_END_DT)
person_season['next_dose_censor_dt'] = person_season['next_vax_date'] - pd.Timedelta(days=1)

# -- Step 4: Censored risk_end_dt = minimum of all candidate censor dates ------
censor_cols = {
    'disenrollment':   person_season['enrl_end_date'],
    'death':           person_season['death_date'],
    'study_end':       person_season['season_end_dt'],
    'observation_end': person_season['risk_end_uncensored'],
    'next_dose':       person_season['next_dose_censor_dt'],
}
censor_df = pd.DataFrame(censor_cols)

person_season['risk_end_dt']   = censor_df.min(axis=1)          # NaT columns skipped
person_season['censor_reason'] = censor_df.idxmin(axis=1)

# Clamp: risk_end_dt cannot precede risk_start_dt (e.g. immediate disenrollment)
person_season['risk_end_dt'] = person_season[['risk_start_dt', 'risk_end_dt']].max(axis=1)

# -- Step 5: Risk length and observed person-time -----------------------------
# Standard: inclusive interval -> +1 day
person_season['risk_length']     = (person_season['risk_end_dt'] - person_season['risk_start_dt']).dt.days + 1
person_season['obs_risk_length'] = person_season['risk_length']   # person-days at risk

# -- Step 6: Outcome flag within risk window ----------------------------------
person_season['outcome_in_risk'] = (
    person_season['myo_date'].notna() &
    (person_season['myo_date'] >= person_season['risk_start_dt']) &
    (person_season['myo_date'] <= person_season['risk_end_dt'])
).astype(int)

# -- Summary -------------------------------------------------------------------
print(f'\nRisk window (days 0-{RISK_WIN_END_DAYS})')
print()
print('Censor reason:')
print(person_season['censor_reason'].value_counts())
print()
print('risk_length distribution (days):')
print(person_season['risk_length'].describe().round(1))
print()
print('outcome_in_risk:')
print(person_season['outcome_in_risk'].value_counts())
print()
print('Persons with myo_after_first_vax=1 whose myo fell OUTSIDE risk window:')
outside = person_season[(person_season['myo_after_first_vax'] == 1) & (person_season['outcome_in_risk'] == 0)]
```

```
print(f'  {len(outside):,}  (censored before myo_date)')
print()
cols_show = ['person_id', 'season', 'first_vax_date', 'risk_end_dt',
             'censor_reason', 'risk_length', 'myo_date', 'outcome_in_risk']
print('Sample rows:')
print(person_season[cols_show].head(10).to_string(index=False))
```

```
Person-season records : 387
Unique persons        : 335

Risk window (days 0-28)

Censor reason:
censor_reason
observation_end    353
study_end           29
death                4
next_dose            1
Name: count, dtype: int64

risk_length distribution (days):
count    387.0
mean      27.6
std        5.1
min        1.0
25%       29.0
50%       29.0
75%       29.0
max       29.0
Name: risk_length, dtype: float64

outcome_in_risk:
outcome_in_risk
0    365
1     22
Name: count, dtype: int64

Persons with myo_after_first_vax=1 whose myo fell OUTSIDE risk window:
  15  (censored before myo_date)

Sample rows:
person_id season first_vax_date risk_end_dt   censor_reason  risk_length   myo_date  outcome_in_risk
  1006810  22_23     2023-02-01  2023-03-01 observation_end           29        NaT                0
  1028374  23_24     2024-04-13  2024-05-11 observation_end           29        NaT                0
  1036161  23_24     2023-08-16  2023-09-13 observation_end           29        NaT                0
  1059486  23_24     2023-08-03  2023-08-31 observation_end           29        NaT                0
  1093026  23_24     2023-08-04  2023-09-01 observation_end           29 2023-10-08                0
  1109031  23_24     2024-06-24  2024-06-30       study_end            7        NaT                0
  1109031  24_25     2024-08-27  2024-09-24 observation_end           29 2024-09-17                1
  1120642  23_24     2024-05-24  2024-06-21 observation_end           29        NaT                0
  1120642  24_25     2024-08-17  2024-09-14 observation_end           29        NaT                0
  1131249  23_24     2023-11-03  2023-12-01 observation_end           29        NaT                0
```

```
# -- Save risk-window / follow-up dataset (person_season.csv) -----------------
_ps_path = DATA_DIR / 'person_season.csv'
try:
    person_season.to_csv(_ps_path, index=False)
    print(f'Saved: person_season.csv  ({len(person_season):,} rows x {person_season.shape[1]} cols)')
except PermissionError:
    print('PermissionError: close person_season.csv in Excel/other app and re-run.')

# -- Preview key risk-window columns -----------------------------------------
_rw_cols = ['person_id', 'season', 'sex', 'age_cat', 'brand',
            'first_vax_date', 'risk_start_dt', 'risk_end_dt',
            'risk_length', 'obs_risk_length', 'censor_reason', 'outcome_in_risk']
_rw_cols = [c for c in _rw_cols if c in person_season.columns]

print()
print('-- Risk-window variable preview (first 10 rows) --')
```

```
print(person_season[_rw_cols].head(10).to_string(index=False))

print()
print('-- Censor reason distribution --')
print(person_season['censor_reason'].value_counts().to_string())

print()
_n_out = int(person_season['outcome_in_risk'].sum())
_n_tot = len(person_season)
print(f'Outcomes in risk window : {_n_out} / {_n_tot} ({_n_out/_n_tot*100:.1f}%)')
print(f'Mean observed risk length: {person_season["obs_risk_length"].mean():.1f} days')
print(f'Total person-days at risk: {int(person_season["obs_risk_length"].sum()):,}')
```

```
Saved: person_season.csv  (387 rows x 29 cols)

-- Risk-window variable preview (first 10 rows) --
person_id season sex age_cat brand first_vax_date risk_start_dt risk_end_dt  risk_length  obs_risk_length   censor_reason  outcome_in_risk
  1006810  22_23   F   12_17   cvm     2023-02-01    2023-02-01  2023-03-01           29               29 observation_end                0
  1028374  23_24   M   18_24   cvm     2024-04-13    2024-04-13  2024-05-11           29               29 observation_end                0
  1036161  23_24   F   12_17   cvp     2023-08-16    2023-08-16  2023-09-13           29               29 observation_end                0
  1059486  23_24   M   12_17   cvn     2023-08-03    2023-08-03  2023-08-31           29               29 observation_end                0
  1093026  23_24   F   18_24   cvp     2023-08-04    2023-08-04  2023-09-01           29               29 observation_end                0
  1109031  23_24   F   12_17   cvp     2024-06-24    2024-06-24  2024-06-30            7                7       study_end                0
  1109031  24_25   F   12_17   cvp     2024-08-27    2024-08-27  2024-09-24           29               29 observation_end                1
  1120642  23_24   M   12_17   cvp     2024-05-24    2024-05-24  2024-06-21           29               29 observation_end                0
  1120642  24_25   M   12_17   cvp     2024-08-17    2024-08-17  2024-09-14           29               29 observation_end                0
  1131249  23_24   M   18_24   cvp     2023-11-03    2023-11-03  2023-12-01           29               29 observation_end                0

-- Censor reason distribution --
censor_reason
observation_end    353
study_end           29
death                4
next_dose            1

Outcomes in risk window : 22 / 387 (5.7%)
Mean observed risk length: 27.6 days
Total person-days at risk: 10,680
```

Part 6 -- Summary Tables

Eight aggregation levels, each saved as a separate CSV. All rate statistics are expressed per **100,000 person-days**.

Variable	Definition
`num_vax`	Number of vaccination records (person-season rows)
`num_unique_person`	Number of distinct individuals
`num_outcomes_risk`	Myocarditis events within the 0-28 day risk window
`pd_risk`	Person-days at risk = `sum(obs_risk_length)`
`rate_risk`	Crude incidence rate = `(num_outcomes_risk / pd_risk) x 100,000`
`rate_se`	Standard error = `sqrt(num_outcomes_risk) / pd_risk x 100,000`
`lower_ci`	Lower 95% CI -- exact Poisson (chi-squared method)
`upper_ci`	Upper 95% CI -- exact Poisson (chi-squared method)

Exact Poisson 95% CI: `lower = chi^2(0.025, 2d) / (2T)`, `upper = chi^2(0.975, 2(d+1)) / (2T)` where d = events, T = person-days. When $d = 0$: lower = 0, upper uses `chi^2(0.975, 2)/(2T)`.

```
from scipy import stats as scipy_stats

# -- Readable brand name --------------------------------------------------------
```

```python
BRAND_MAP = {'cvp': 'pfizer', 'cvm': 'moderna', 'cvn': 'novavax', 'cvj': 'janssen'}
person_season['brand_name'] = person_season['brand'].map(BRAND_MAP)

# -- Core statistics function (person-days as denominator) ---------------------
def compute_stats(subdf):
    d  = int(subdf['outcome_in_risk'].sum())
    T  = subdf['obs_risk_length'].sum()
    if T > 0:
        rate = d / T * 100_000
        se   = d**0.5 / T * 100_000
        lo = (scipy_stats.chi2.ppf(0.025, 2 * d)      / (2 * T) * 100_000) if d > 0 else 0.0
        hi =  scipy_stats.chi2.ppf(0.975, 2 * (d + 1)) / (2 * T) * 100_000
    else:
        rate = se = lo = hi = float('nan')
    return {
        'num_vax':           len(subdf),
        'num_unique_person': subdf['person_id'].nunique(),
        'num_outcomes_risk': d,
        'pd_risk':           int(T),
        'rate_risk':         round(rate, 4) if pd.notna(rate) else float('nan'),
        'lower_ci':          round(lo,   4) if pd.notna(lo)   else float('nan'),
        'upper_ci':          round(hi,   4) if pd.notna(hi)   else float('nan'),
        'rate_se':           round(se,   4) if pd.notna(se)   else float('nan'),
    }

def summarize_by(df, group_cols):
    if not group_cols:
        return pd.DataFrame([compute_stats(df)])
    rows = []
    for keys, grp in df.groupby(group_cols, sort=True):
        if not isinstance(keys, tuple):
            keys = (keys,)
        row = dict(zip(group_cols, keys))
        row.update(compute_stats(grp))
        rows.append(row)
    return pd.DataFrame(rows)

# -- Run all 11 aggregations --------------------------------------------------
AGGREGATIONS = {
    'season_brand_age': ['season', 'brand_name', 'age_cat'],
    'season_brand':     ['season', 'brand_name'],
    'season_age':       ['season', 'age_cat'],
    'season_sex':       ['season', 'sex'],
    'season':           ['season'],
    'brand_age':        ['brand_name', 'age_cat'],
    'brand_sex':        ['brand_name', 'sex'],
    'brand':            ['brand_name'],
    'age':              ['age_cat'],
    'sex':              ['sex'],
    'overall':          [],
}

all_summaries = {}
print(f'{"Aggregation":<20}  {"Groups":>6}  {"Outcomes":>9}  {"PD at risk":>12}')
print('-' * 55)

for agg_name, group_cols in AGGREGATIONS.items():
    df_out = summarize_by(person_season, group_cols)
    all_summaries[agg_name] = df_out
    out_path = DATA_DIR / f'summary_{agg_name}.csv'
    df_out.to_csv(out_path, index=False)
    print(f'{agg_name:<20}  {len(df_out):>6}  {int(df_out["num_outcomes_risk"].sum()):>9}  {int(df_out["pd_risk"].sum()):>12,}')

# -- Display key tables -------------------------------------------------------
print()
print('--- Overall -----------------------------------------------------')
print(all_summaries['overall'].to_string(index=False))

print()
```

```
print('--- By season ----------------------------------------------------')
print(all_summaries['season'].to_string(index=False))

print()
print('--- By brand -----------------------------------------------------')
print(all_summaries['brand'].to_string(index=False))

print()
print('--- By age -------------------------------------------------------')
print(all_summaries['age'].to_string(index=False))

print()
print('--- By sex -------------------------------------------------------')
print(all_summaries['sex'].to_string(index=False))

print()
print('--- Season x brand -----------------------------------------------')
print(all_summaries['season_brand'].to_string(index=False))

print()
print('--- Season x sex -------------------------------------------------')
print(all_summaries['season_sex'].to_string(index=False))

print()
print('--- Brand x sex --------------------------------------------------')
print(all_summaries['brand_sex'].to_string(index=False))
```

Aggregation	Groups	Outcomes	PD at risk
season_brand_age	19	22	10,680
season_brand	10	22	10,680
season_age	6	22	10,680
season_sex	6	22	10,680
season	3	22	10,680
brand_age	7	22	10,680
brand_sex	8	22	10,680
brand	4	22	10,680
age	2	22	10,680
sex	2	22	10,680
overall	1	22	10,680

--- Overall --

num_vax	num_unique_person	num_outcomes_risk	pd_risk	rate_risk	lower_ci	upper_ci	rate_se
387	335	22	10680	205.9925	129.0944	311.8751	43.9178

--- By season --

season	num_vax	num_unique_person	num_outcomes_risk	pd_risk	rate_risk	lower_ci	upper_ci	rate_se
22_23	110	110	3	3051	98.3284	20.2777	287.3574	56.7699
23_24	183	183	10	4960	201.6129	96.6812	370.7733	63.7556
24_25	94	94	9	2669	337.2049	154.1916	640.1200	112.4016

--- By brand ---

brand_name	num_vax	num_unique_person	num_outcomes_risk	pd_risk	rate_risk	lower_ci	upper_ci	rate_se
janssen	8	8	0	232	0.0000	0.0000	1590.0342	0.0000
moderna	144	131	8	3970	201.5113	86.9983	397.0577	71.2450
novavax	121	107	6	3324	180.5054	66.2423	392.8843	73.6910
pfizer	114	102	8	3154	253.6462	109.5064	499.7841	89.6775

--- By age ---

age_cat	num_vax	num_unique_person	num_outcomes_risk	pd_risk	rate_risk	lower_ci	upper_ci	rate_se
12_17	146	126	10	4065	246.0025	117.9677	452.4073	77.7928
18_24	241	211	12	6615	181.4059	93.7351	316.8796	52.3674

--- By sex ---

sex	num_vax	num_unique_person	num_outcomes_risk	pd_risk	rate_risk	lower_ci	upper_ci	rate_se
F	195	168	12	5295	226.6289	117.1025	395.8751	65.4221
M	192	167	10	5385	185.7010	89.0509	341.5108	58.7238

--- Season x brand ---

season	brand_name	num_vax	num_unique_person	num_outcomes_risk	pd_risk	rate_risk	lower_ci	upper_ci	rate_se
22_23	janssen	8	8	0	232	0.0000	0.0000	1590.0342	0.0000
22_23	moderna	39	39	1	1068	93.6330	2.3706	521.6895	93.6330
22_23	novavax	32	32	1	866	115.4734	2.9235	643.3768	115.4734
22_23	pfizer	31	31	1	885	112.9944	2.8608	629.5642	112.9944
23_24	moderna	69	69	3	1859	161.3771	33.2798	471.6123	93.1711
23_24	novavax	57	57	2	1558	128.3697	15.5462	463.7155	90.7711
23_24	pfizer	57	57	5	1543	324.0441	105.2162	756.2108	144.9169
24_25	moderna	36	36	4	1043	383.5091	104.4933	981.9356	191.7546
24_25	novavax	32	32	3	900	333.3333	68.7413	974.1415	192.4501
24_25	pfizer	26	26	2	726	275.4821	33.3622	995.1360	194.7953

--- Season x sex ---

season	sex	num_vax	num_unique_person	num_outcomes_risk	pd_risk	rate_risk	lower_ci	upper_ci	rate_se
22_23	F	52	52	1	1445	69.2042	1.7521	385.5809	69.2042
22_23	M	58	58	2	1606	124.5330	15.0815	449.8560	88.0581
23_24	F	95	95	7	2487	281.4636	113.1630	579.9226	106.3832
23_24	M	88	88	3	2473	121.3101	25.0171	354.5197	70.0384
24_25	F	48	48	4	1363	293.4703	79.9608	751.4005	146.7351
24_25	M	46	46	5	1306	382.8484	124.3098	893.4404	171.2150

--- Brand x sex --

brand_name	sex	num_vax	num_unique_person	num_outcomes_risk	pd_risk	rate_risk	lower_ci	upper_ci	rate_se
janssen	F	6	6	0	174	0.0000	0.0000	2120.0457	0.0000
janssen	M	2	2	0	58	0.0000	0.0000	6360.1370	0.0000
moderna	F	67	62	5	1835	272.4796	88.4734	635.8764	121.8566
moderna	M	77	69	3	2135	140.5152	28.9776	410.6451	81.1265

```
    novavax   F       62                 56                 3     1666   180.0720   37.1352  526.2469 103.9646
    novavax   M       59                 51                 3     1658   180.9409   37.3144  528.7861 104.4663
     pfizer   F       60                 54                 4     1620   246.9136   67.2756  632.1968 123.4568
     pfizer   M       54                 48                 4     1534   260.7562   71.0473  667.6394 130.3781
```

```python
# Save combined all_summaries (all 11 aggregation levels in one file).
# person_season.csv is saved in the Part 5 cell above.

_rows = []
for _lvl, _df in all_summaries.items():
    _tmp = _df.copy()
    _tmp.insert(0, 'aggregation_level', _lvl)
    _tmp['season'] = _tmp['season']      if 'season'     in _tmp.columns else 'All'
    _tmp['brand']  = _tmp['brand_name']  if 'brand_name' in _tmp.columns else 'All'
    _tmp['age']    = _tmp['age_cat']     if 'age_cat'    in _tmp.columns else 'All'
    _tmp['sex']    = _tmp['sex']         if 'sex'        in _tmp.columns else 'All'
    _rows.append(_tmp)

all_summaries_combined = pd.concat(_rows, ignore_index=True, sort=False)

for _col in ['season', 'brand', 'age', 'sex']:
    all_summaries_combined[_col] = all_summaries_combined[_col].fillna('All')

_stat_cols = ['num_vax', 'num_unique_person', 'num_outcomes_risk',
              'pd_risk', 'rate_risk', 'lower_ci', 'upper_ci', 'rate_se']
all_summaries_combined = all_summaries_combined[
    ['aggregation_level', 'season', 'brand', 'age', 'sex'] + _stat_cols
]

_comb_path = DATA_DIR / 'all_summaries_combined.csv'
try:
    all_summaries_combined.to_csv(_comb_path, index=False)
    print(f'Saved: all_summaries_combined.csv  ({len(all_summaries_combined):,} rows x {all_summaries_combined.shape[1]} cols)')
except PermissionError:
    print('PermissionError: close all_summaries_combined.csv and re-run.')

print()
print(all_summaries_combined.to_string(index=False))
```

Saved: all_summaries_combined.csv (68 rows x 13 cols)

aggregation_level	season	brand	age	sex	num_vax	num_unique_person	num_outcomes_risk	pd_risk	rate_risk	lower_ci	upper_ci	rate_se
season_brand_age	22_23	janssen	18_24	All	8	8	0	232	0.0000	0.0000	1590.0342	0.0000
season_brand_age	22_23	moderna	12_17	All	14	14	1	384	260.4167	6.5932	1450.9488	260.4167
season_brand_age	22_23	moderna	18_24	All	25	25	0	684	0.0000	0.0000	539.3099	0.0000
season_brand_age	22_23	novavax	12_17	All	10	10	0	290	0.0000	0.0000	1272.0274	0.0000
season_brand_age	22_23	novavax	18_24	All	22	22	1	576	173.6111	4.3955	967.2992	173.6111
season_brand_age	22_23	pfizer	12_17	All	7	7	0	189	0.0000	0.0000	1951.7881	0.0000
season_brand_age	22_23	pfizer	18_24	All	24	24	1	696	143.6782	3.6376	800.5235	143.6782
season_brand_age	23_24	moderna	12_17	All	36	36	1	998	100.2004	2.5369	558.2809	100.2004
season_brand_age	23_24	moderna	18_24	All	33	33	2	861	232.2880	28.1312	839.1043	164.2524
season_brand_age	23_24	novavax	12_17	All	19	19	0	515	0.0000	0.0000	716.2873	0.0000
season_brand_age	23_24	novavax	18_24	All	38	38	2	1043	191.7546	23.2224	692.6834	135.5909
season_brand_age	23_24	pfizer	12_17	All	20	20	2	530	377.3585	45.6999	1363.1486	266.8327
season_brand_age	23_24	pfizer	18_24	All	37	37	3	1013	296.1500	61.0733	865.4761	170.9823
season_brand_age	24_25	moderna	12_17	All	18	18	4	521	767.7543	209.1872	1965.7560	383.8772
season_brand_age	24_25	moderna	18_24	All	18	18	0	522	0.0000	0.0000	706.6819	0.0000
season_brand_age	24_25	novavax	12_17	All	12	12	1	348	287.3563	7.2752	1601.0470	287.3563
season_brand_age	24_25	novavax	18_24	All	20	20	2	552	362.3188	43.8785	1308.8202	256.1981
season_brand_age	24_25	pfizer	12_17	All	10	10	1	290	344.8276	8.7303	1921.2563	344.8276
season_brand_age	24_25	pfizer	18_24	All	16	16	1	436	229.3578	5.8068	1277.8999	229.3578
season_brand	22_23	janssen	All	All	8	8	0	232	0.0000	0.0000	1590.0342	0.0000
season_brand	22_23	moderna	All	All	39	39	1	1068	93.6330	2.3706	521.6895	93.6330
season_brand	22_23	novavax	All	All	32	32	1	866	115.4734	2.9235	643.3768	115.4734
season_brand	22_23	pfizer	All	All	31	31	1	885	112.9944	2.8608	629.5642	112.9944
season_brand	23_24	moderna	All	All	69	69	3	1859	161.3771	33.2798	471.6123	93.1711
season_brand	23_24	novavax	All	All	57	57	2	1558	128.3697	15.5462	463.7155	90.7711
season_brand	23_24	pfizer	All	All	57	57	5	1543	324.0441	105.2162	756.2108	144.9169
season_brand	24_25	moderna	All	All	36	36	4	1043	383.5091	104.4933	981.9356	191.7546
season_brand	24_25	novavax	All	All	32	32	3	900	333.3333	68.7413	974.1415	192.4501
season_brand	24_25	pfizer	All	All	26	26	2	726	275.4821	33.3622	995.1360	194.7953
season_age	22_23	All	12_17	All	31	31	1	863	115.8749	2.9337	645.6134	115.8749
season_age	22_23	All	18_24	All	79	79	2	2188	91.4077	11.0699	330.1960	64.6350
season_age	23_24	All	12_17	All	75	75	3	2043	146.8429	30.2825	429.1372	84.7798
season_age	23_24	All	18_24	All	108	108	7	2917	239.9726	96.4814	494.4352	90.7011
season_age	24_25	All	12_17	All	40	40	6	1159	517.6877	189.9822	1126.7881	211.3451
season_age	24_25	All	18_24	All	54	54	3	1510	198.6755	40.9717	580.6141	114.7054
season_sex	22_23	All	All	F	52	52	1	1445	69.2042	1.7521	385.5809	69.2042
season_sex	22_23	All	All	M	58	58	2	1606	124.5330	15.0815	449.8560	88.0581
season_sex	23_24	All	All	F	95	95	7	2487	281.4636	113.1630	579.9226	106.3832
season_sex	23_24	All	All	M	88	88	3	2473	121.3101	25.0171	354.5197	70.0384
season_sex	24_25	All	All	F	48	48	4	1363	293.4703	79.9608	751.4005	146.7351
season_sex	24_25	All	All	M	46	46	5	1306	382.8484	124.3098	893.4404	171.2150
season	22_23	All	All	All	110	110	3	3051	98.3284	20.2777	287.3574	56.7699
season	23_24	All	All	All	183	183	10	4960	201.6129	96.6812	370.7733	63.7556
season	24_25	All	All	All	94	94	9	2669	337.2049	154.1916	640.1200	112.4016
brand_age	All	janssen	18_24	All	8	8	0	232	0.0000	0.0000	1590.0342	0.0000
brand_age	All	moderna	12_17	All	68	59	6	1903	315.2916	115.7065	686.2572	128.7173
brand_age	All	moderna	18_24	All	76	72	2	2067	96.7586	11.7179	349.5253	68.4187
brand_age	All	novavax	12_17	All	41	37	1	1153	86.7303	2.1958	483.2301	86.7303
brand_age	All	novavax	18_24	All	80	71	5	2171	230.3086	74.7806	537.4635	102.9971
brand_age	All	pfizer	12_17	All	37	31	3	1009	297.3241	61.3154	868.9071	171.6601
brand_age	All	pfizer	18_24	All	77	71	5	2145	233.1002	75.6870	543.9782	104.2456
brand_sex	All	janssen	All	F	6	6	0	174	0.0000	0.0000	2120.0457	0.0000
brand_sex	All	janssen	All	M	2	2	0	58	0.0000	0.0000	6360.1370	0.0000
brand_sex	All	moderna	All	F	67	62	5	1835	272.4796	88.4734	635.8764	121.8566
brand_sex	All	moderna	All	M	77	69	3	2135	140.5152	28.9776	410.6451	81.1265
brand_sex	All	novavax	All	F	62	56	3	1666	180.0720	37.1352	526.2469	103.9646
brand_sex	All	novavax	All	M	59	51	3	1658	180.9409	37.3144	528.7861	104.4663
brand_sex	All	pfizer	All	F	60	54	4	1620	246.9136	67.2756	632.1968	123.4568
brand_sex	All	pfizer	All	M	54	48	4	1534	260.7562	71.0473	667.6394	130.3781
brand	All	janssen	All	All	8	8	0	232	0.0000	0.0000	1590.0342	0.0000
brand	All	moderna	All	All	144	131	8	3970	201.5113	86.9983	397.0577	71.2450
brand	All	novavax	All	All	121	107	6	3324	180.5054	66.2423	392.8843	73.6910
brand	All	pfizer	All	All	114	102	8	3154	253.6462	109.5064	499.7841	89.6775
age	All	All	12_17	All	146	126	10	4065	246.0025	117.9677	452.4073	77.7928
age	All	All	18_24	All	241	211	12	6615	181.4059	93.7351	316.8796	52.3674
sex	All	All	All	F	195	168	12	5295	226.6289	117.1025	395.8751	65.4221

```
      sex    All     All   All   M      192               167               10     5385   185.7010   89.0509  341.5108  58.7238
  overall    All     All   All All      387               335               22    10680   205.9925  129.0944  311.8751  43.9178
```

Key Findings and Conclusions

Cohort Overview

The analytic cohort comprised **335 unique individuals** (387 person-season vaccination records) aged 12-24 years who received a COVID-19 vaccine dose within the study window (2022-2026) and met all inclusion criteria. A total of **22 myopericarditis events** were observed within the 28-day risk window, yielding 10,680 person-days at risk.

---

1. Overall Incidence

The crude myopericarditis incidence rate across all strata was **205.99 per 100,000 person-days** (95% CI: 129.09-311.88). This overall estimate serves as the reference point for all stratified comparisons below.

---

2. Seasonal Variation

Season	Events	Rate (per 100k PD)	95% CI
22_23	3	98.33	20.28 - 287.36
23_24	10	201.61	96.68 - 370.77
24_25	9	337.20	154.19 - 640.12

Finding: A pronounced upward trend in incidence is observed across COVID19-year seasons. The rate more than **tripled** from season 22_23 (98.3/100k PD) to 24_25 (337.2/100k PD). While confidence intervals widen with smaller denominators in the most recent season, the 22_23 interval is clearly below those of 23_24 and 24_25, suggesting a genuine increase in risk or ascertainment over time. This escalating pattern is the primary seasonal signal and should be examined for underlying drivers (variant circulation, booster campaigns, surveillance intensity) in downstream analyses.

---

3. Vaccine Brand

Brand	Events	Rate (per 100k PD)	95% CI
Pfizer	8	253.65	109.51 - 499.78
Moderna	8	201.51	87.00 - 397.06
Novavax	6	180.51	66.24 - 392.88
Janssen	0	0.00	0.00 - 1590.03

Finding: Among the three mRNA-adjacent and protein-subunit brands with observed events, Pfizer carried the highest point estimate (253.6/100k PD), followed by Moderna (201.5) and Novavax (180.5). Confidence intervals overlap substantially across all three, precluding firm brand-level inference at this sample size. Janssen had only 8 records and zero events; its wide upper CI (1590/100k PD) reflects this sparsity. Brand-specific comparisons will require formal regression adjustment in subsequent analyses.

---

4. Age Group

Age group	Events	Rate (per 100k PD)	95% CI
12-17	10	246.00	117.97 - 452.41
18-24	12	181.41	93.74 - 316.88

Finding: Adolescents aged 12-17 had a higher point estimate (246.0/100k PD) than young adults aged 18-24 (181.4/100k PD), consistent with the literature showing greater myocarditis signal in younger age groups following mRNA vaccination. The confidence intervals overlap, and the difference is not statistically evaluated here; formal comparison is reserved for the regression stage.

---

5. Sex

Sex	Events	Rate (per 100k PD)	95% CI
Female	12	226.63	117.10 - 395.88
Male	10	185.70	89.05 - 341.51

Finding: Females showed a marginally higher point estimate (226.6/100k PD) than males (185.7/100k PD), though the confidence intervals overlap broadly. This direction is noteworthy given that real-world surveillance has predominantly detected a stronger myocarditis signal in males (particularly adolescent males after mRNA vaccines); the difference here likely reflects the synthetic nature of the data rather than a true biological pattern.

---

6. Season x Sex Interaction

Season	Female rate	Male rate	Difference
22_23	69.2	124.5	Males higher
23_24	281.5	121.3	Females notably higher
24_25	293.5	382.8	Males higher

Finding: The sex-season interaction reveals heterogeneous patterns across seasons. In 23_24, females had more than double the male rate (281.5 vs 121.3/100k PD). In 24_25 the pattern reversed with males higher (382.8 vs 293.5). The 22_23 season showed low rates in both sexes. The inconsistency across seasons suggests either chance variation (small cell sizes) or true effect modification that warrants formal testing.

Disclaimer

> **This analysis is based entirely on synthetic data** generated to replicate the structural characteristics of real-world administrative insurance claims, and **does not represent true clinical or epidemiological evidence.**
>
> The incidence rates, confidence intervals, and stratified patterns reported here are an artefact of the data generation process and should not be interpreted as reflecting the actual risk of myopericarditis following COVID-19 vaccination in any real-world population. In particular:
>
> - Observed rates and trends may differ substantially from those reported in published pharmacoepidemiological studies using genuine claims or registry data.
> - Subgroup differences (by season, brand, age, or sex) carry no inferential weight and should not be used to guide clinical practice, vaccination policy, or regulatory decision-making.
> - The synthetic dataset was designed solely to validate the analytic pipeline and to serve as a reproducible demonstration environment.
>
> All substantive conclusions regarding the safety profile of COVID-19 vaccines must be drawn from peer-reviewed analyses conducted on verified real-world data under appropriate ethical and regulatory oversight.